Analysis of Algorithm (CSN13400)
ASSIGNMENT-1

Name : Aayansh Niranjan
Reg_No.: 20253004
Date :26/07/2026

Q1. Write a program to insert a node in the middle of a Singly Linked List.

A1. CODE:

```
#include<stdio.h>
#include<stdlib.h>

struct node{
  int val;
  struct node* next;
};

void printList(struct node *head) {
    struct node *temp=head;
    while(temp){
        printf("%d ",temp->val);
        temp=temp->next;
    }
    printf("\n");
}

int main(){
  struct node* head=(struct node*)malloc(sizeof(struct node));
  struct node* mover=head;
  for(int i=1;i<10;i++){
    struct node* temp=(struct node*)malloc(sizeof(struct node));
    temp->val=i;
    mover->next=temp;
    mover=mover->next;
  }
  int n=5;
  int val;
  scanf("%d",&val);
  mover=head;
  while(n--){
    mover=mover->next;
  }
  struct node* temp=(struct node*)malloc(sizeof(struct node));
  temp->val=val;
  temp->next=mover->next;
  mover->next=temp;
  printList(head->next);
}
```

OUTPUT:

```
rithm/Assignment-1$ ./p1
4
1 2 3 4 5 4 6 7 8 9
```

Q2. Write a program to remove duplicates from a single unsorted linked list.
A2. CODE:

```
#include<stdio.h>
#include<stdlib.h>

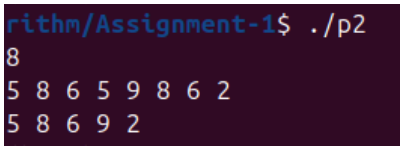
struct node{
   int val;
   struct node*next;
};

struct node*createNode(int val){
   struct node*temp=(struct node*)malloc(sizeof(struct node));
   temp->val=val;
   temp->next=NULL;
   return temp;
}

void printList(struct node*head){
   while(head!=NULL){
      printf("%d ",head->val);
      head=head->next;
   }
   printf("\n");
}

int main(){
   int n;
   scanf("%d",&n);
   struct node*head=(struct node*)malloc(sizeof(struct node));
   head->next=NULL;
   struct node*tail=head;
   for(int i=0;i<n;i++){
      int x;
      scanf("%d",&x);
      struct node*temp=createNode(x);
      tail->next=temp;
      tail=temp;
   }
   struct node*curr=head->next;
   while(curr!=NULL){
      struct node*prev=curr;
      struct node*nextNode=curr->next;
      while(nextNode!=NULL){
         if(nextNode->val==curr->val){
            prev->next=nextNode->next;
            free(nextNode);
            nextNode=prev->next;
         }
         else{
            prev=nextNode;
            nextNode=nextNode->next;
         }
      }
      curr=curr->next;
   }
   printList(head->next);
   return 0;
}
```

OUTPUT:



Q3. Write a C program to find a pair in a singly linked list whose sum is equal to a given value.
A3. CODE:

```
#include<stdio.h>
#include<stdlib.h>

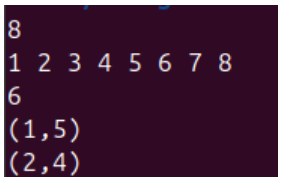
struct node{
   int val;
   struct node*next;
};

struct node*createNode(int val){
   struct node*temp=(struct node*)malloc(sizeof(struct node));
   temp->val=val;
   temp->next=NULL;
   return temp;
}

int main(){
   int n,x,sum;
   scanf("%d",&n);
   struct node*head=(struct node*)malloc(sizeof(struct node));
   head->next=NULL;
   struct node*tail=head;
   while(n--){
      scanf("%d",&x);
      struct node*temp=createNode(x);
      tail->next=temp;
      tail=temp;
   }
   scanf("%d",&sum);
   int found=0;
   for(struct node*i=head->next;i;i=i->next){
      for(struct node*j=i->next;j;j=j->next){
         if(i->val+j->val==sum){
            printf("(%d,%d)\n",i->val,j->val);
            found=1;
         }
      }
   }
   if(!found){printf("No pair found");}

   return 0;
}
```

OUTPUT:



Q4.Write a C program to detect whether a given singly linked list contains a loop or not.
A4. CODE:

```
#include<stdio.h>
#include<stdlib.h>

struct node{
    int val;
    struct node*next;
};

struct node*createNode(int val){
    struct node*temp=(struct node*)malloc(sizeof(struct node));
    temp->val=val;
    temp->next=NULL;
    return temp;
}

int main(){
    int n,x,pos;
    scanf("%d",&n);
    struct node*head=(struct node*)malloc(sizeof(struct node));
    head->next=NULL;
    struct node*tail=head,*loopNode=NULL;
    for(int i=1;i<=n;i++){
        scanf("%d",&x);
        struct node*temp=createNode(x);
        tail->next=temp;
        tail=temp;
        if(i==1){loopNode=temp;}
    }
    scanf("%d",&pos);
    if(pos!=0){
        loopNode=head->next;
        for(int i=1;i<pos;i++){loopNode=loopNode->next;}
        tail->next=loopNode;
    }
    struct node*slow=head->next;
    struct node*fast=head->next;
    while(fast&&fast->next){
        slow=slow->next;
        fast=fast->next->next;
        if(slow==fast){
            printf("Loop detected");
            return 0;
        }
    }
    printf("No loop");
    return 0;
}
```

OUTPUT:

```
9
1 2 3 4 5 6 7 8 9
5
Loop detected(base) User@ccsf-H
```

```
9
1 2 3 4 5 6 7 8 9
0
No loop(base) User@ccsf-HP-Pro-
```

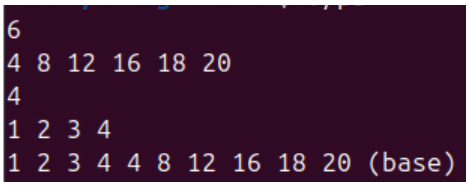
Q5. Merge two sorted singly linked lists into one sorted linked list.
A5. CODE:

```
#include<stdio.h>
#include<stdlib.h>

struct node{int val;struct node*next;};
struct node*createNode(int val){
   struct node*temp=(struct node*)malloc(sizeof(struct node));
   temp->val=val;
   temp->next=NULL;
   return temp;
}
void printList(struct node*head){
   while(head){
      printf("%d ",head->val);
      head=head->next;
   }
}
struct node*merge(struct node*l1,struct node*l2){
   struct node*head=(struct node*)malloc(sizeof(struct node));
   head->next=NULL;
   struct node*tail=head;
   while(l1&&l2){
      if(l1->val<=l2->val){tail->next=l1;l1=l1->next;}
      else{tail->next=l2;l2=l2->next;}
      tail=tail->next;
   }
   if(l1) tail->next=l1;
   else tail->next=l2;
   return head->next;
}

int main(){
   int n,m,x;
   scanf("%d",&n);
   struct node*h1=(struct node*)malloc(sizeof(struct node));
   h1->next=NULL;
   struct node*t1=h1;
   while(n--){
      scanf("%d",&x);
      t1->next=createNode(x);
      t1=t1->next;
   }
   scanf("%d",&m);
   struct node*h2=(struct node*)malloc(sizeof(struct node));
   h2->next=NULL;
   struct node*t2=h2;
   while(m--){
      scanf("%d",&x);
      t2->next=createNode(x);
      t2=t2->next;
   }
   struct node*ans=merge(h1->next,h2->next);
   printList(ans);
   return 0;
}
```

OUTPUT:

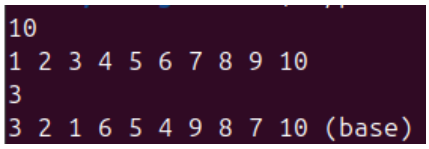


Q6. Given a linked list, reverse it in groups of size k.
A6. CODE:

```
#include<stdio.h>
#include<stdlib.h>
struct node{int val;struct node*next;};
struct node*createNode(int val){
    struct node*temp=(struct node*)malloc(sizeof(struct node));
    temp->val=val;
    temp->next=NULL;
    return temp;
}
void printList(struct node*head){
    while(head){printf("%d ",head->val);head=head->next;}
}
struct node*reverseK(struct node*head,int k){
    struct node*dummy=(struct node*)malloc(sizeof(struct node));
    dummy->next=head;
    struct node*prev=dummy,*curr,*next;
    struct node*check;
    while(1){
        check=prev;
        for(int i=0;i<k&&check;i++){check=check->next;}
        if(!check){break;}
        curr=prev->next;
        next=curr->next;
        for(int i=1;i<k;i++){
            curr->next=next->next;
            next->next=prev->next;
            prev->next=next;
            next=curr->next;
        }
        prev=curr;
    }
    return dummy->next;
}

int main(){
    int n,k,x;
    scanf("%d",&n);
    struct node*head=(struct node*)malloc(sizeof(struct node));
    head->next=NULL;
    struct node*tail=head;
    while(n--){
        scanf("%d",&x);
        tail->next=createNode(x);
        tail=tail->next;
    }
    scanf("%d",&k);
    head->next=reverseK(head->next,k);
    printList(head->next);
    return 0;
}
```

OUTPUT:



Q7. Write a program to implement a stack using two queues. You are only allowed to use the standard queue operations.

A7. CODE:

```
#include<stdio.h>

int q1[100],q2[100];
int f1=0,r1=-1,f2=0,r2=-1;

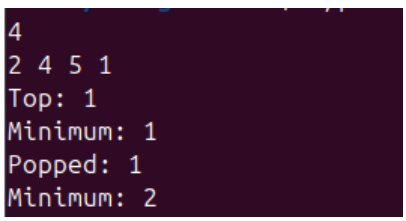
void enqueue(int q[],int *r,int x){q[++(*r)]=x;}
int dequeue(int q[],int *f){return q[(*f)++];}
int empty(int f,int r){return f>r;}
void push(int x){
   enqueue(q2,&r2,x);
   while(!empty(f1,r1)){enqueue(q2,&r2,dequeue(q1,&f1));}
   for(int i=0;i<=r2;i++){q1[i]=q2[i];}
   f1=f2;r1=r2;f2=0;r2=-1;
}

int pop(){
   if(empty(f1,r1)){printf("Stack empty\n");return -1;}
   return dequeue(q1,&f1);
}

int peek(){
   if(empty(f1,r1)){printf("Stack empty\n");return -1;}
   return q1[f1];
}

int main(){
   int n,x;
   scanf("%d",&n);
   while(n--){
      scanf("%d",&x);
      push(x);
   }
   printf("Top: %d\n",peek());
   printf("Pop: %d\n",pop());
   printf("Pop: %d\n",pop());
   printf("Top: %d\n",peek());
   return 0;
}
```

OUTPUT:



Q8. Design a stack that supports push(), pop(), top(), and getMin() operations in O(1) time.

A8. CODE:

```
#include<stdio.h>

#define MAX 100

int stack[MAX],minStack[MAX];
int top=-1,minTop=-1;

void push(int x){
   stack[++top]=x;
   if(minTop==-1 || x<=minStack[minTop]){minStack[++minTop]=x;}
}

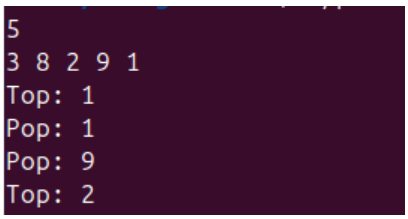
int pop(){
   if(top==-1){printf("Stack empty\n");return -1;}
   int x=stack[top--];
   if(x==minStack[minTop]){minTop--;}
   return x;
}

int peek(){
   if(top==-1){printf("Stack empty\n");return -1;}
   return stack[top];
}

int getMin(){
   if(minTop==-1){printf("Stack empty\n");return -1;}
   return minStack[minTop];
}

int main(){
   int n,x;
   scanf("%d",&n);
   while(n--){
      scanf("%d",&x);
      push(x);
   }
   printf("Top: %d\n",peek());
   printf("Minimum: %d\n",getMin());
   printf("Popped: %d\n",pop());
   printf("Minimum: %d\n",getMin());
   return 0;
}
```

OUTPUT:



Q9. Write a function to convert an infix expression (e.g., (a+b)*c) to postfix (e.g., ab+c*) using a stack. Further, write a function to evaluate a postfix expression using a stack.

A9. CODE:

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>

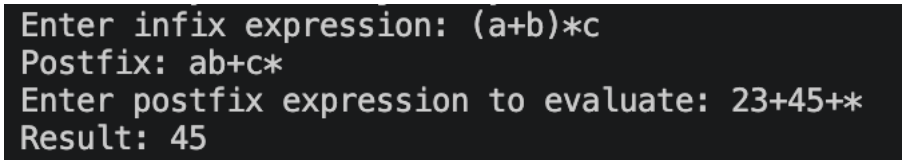
char stack[100];
int top=-1;
void push(char x){stack[++top]=x;}
char pop(){return stack[top--];}
char peek(){return stack[top];}
int precedence(char x){
   if(x=='^')return 3;
   if(x=='*'||x=='/')return 2;
   if(x=='+'||x=='-')return 1;
   return 0;
}

void infixToPostfix(char infix[],char postfix[]){
   int i,j=0;
   top=-1;
   for(i=0;infix[i];i++){
      char ch=infix[i];
      if(isalnum(ch)){postfix[j++]=ch;}
      else if(ch=='('){push(ch);}
      else if(ch==')'){
         while(top!=-1&&peek()!='('){postfix[j++]=pop();}
         pop();
      }
      else{
         while(top!=-1&&precedence(peek())>=precedence(ch)){postfix[j++]=pop();}
         push(ch);
      }
   }
   while(top!=-1){postfix[j++]=pop();}
   postfix[j]='\0';
}
int valStack[100],vtop=-1;
void pushVal(int x){{valStack[++vtop]=x;}}
int popVal(){{return valStack[vtop--];}}
int evaluatePostfix(char postfix[]){
   int i;
   vtop=-1;
   for(i=0;postfix[i];i++){
      char ch=postfix[i];
      if(isdigit(ch)){pushVal(ch-'0');}
      else{
         int b=popVal();
         int a=popVal();
         switch(ch){
            case '+':pushVal(a+b);break;
            case '-':pushVal(a-b);break;
            case '*':pushVal(a*b);break;
            case '/':pushVal(a/b);break;
```

```
            case '^':{
                int p=1,j;
                for(j=0;j<b;j++)p*=a;
                pushVal(p);
                break;
              }
            }
        }
    }
    return popVal();
}

int main(){
    char infix[100],postfix[100];
    printf("Enter infix expression: ");
    scanf("%s",infix);
    infixToPostfix(infix,postfix);
    printf("Postfix: %s\n",postfix);
    printf("Enter postfix expression to evaluate: ");
    scanf("%s",postfix);
    printf("Result: %d\n",evaluatePostfix(postfix));
    return 0;
}
```

OUTPUT:



Q10. Given a queue and an integer k, reverse the first k elements of the queue while leaving the rest of the elements in the same order.

A10. CODE:

```
#include <stdio.h>
int queue[100];
int front=0,rear=-1,size=0;

void enqueue(int x){
    if(size==100)return;
    rear=(rear+1)%100;
    queue[rear]=x;
    size++;
}

int dequeue(){
    int x=queue[front];
    front=(front+1)%100;
    size--;
    return x;
}

void display(){
    int i,idx=front;
    for(i=0;i<size;i++){
        printf("%d ",queue[idx]);
        idx=(idx+1)%100;
    }
    printf("\n");
}

void reverseFirstK(int k){
    int stack[100],top=-1;
    int i,n=size;
    for(i=0;i<k;i++){stack[++top]=dequeue();}
    while(top!=-1){enqueue(stack[top--]);}
    for(i=0;i<n-k;i++){enqueue(dequeue());}
}

int main(){
    int n,k,i,x;
    printf("Enter number of elements: ");
    scanf("%d",&n);
    printf("Enter queue elements: ");
    for(i=0;i<n;i++){scanf("%d",&x);enqueue(x);}
    printf("Enter k: ");
    scanf("%d",&k);
    reverseFirstK(k);
    printf("Queue after reversing first %d elements: ",k);
    display();
    return 0;
}
```

OUTPUT:

```
Enter number of elements: 5
Enter queue elements: 10 20 30 40 50
Enter k: 3
Queue after reversing first 3 elements: 30 20 10 40 50
```

Q11.Write a C program to create a Binary Tree and perform the following tree traversal operations: Inorder Traversal, Preorder Traversal, Postorder Traversal. Each traversal should be implemented as a recursive function.

A11. CODE:

```
#include <stdio.h>
#include <stdlib.h>

struct Node{
    int data;
    struct Node *left,*right;
};

struct Node* createNode(int data){
    struct Node *newNode=(struct Node*)malloc(sizeof(struct Node));
    newNode->data=data;
    newNode->left=NULL;
    newNode->right=NULL;
    return newNode;
}

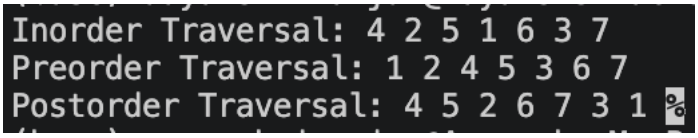
void inorder(struct Node *root){
    if(root==NULL){return;}
    inorder(root->left);
    printf("%d ",root->data);
    inorder(root->right);
}

void preorder(struct Node *root){
    if(root==NULL){return;}
    printf("%d ",root->data);
    preorder(root->left);
    preorder(root->right);
}

void postorder(struct Node *root){
    if(root==NULL){return;}
    postorder(root->left);
    postorder(root->right);
    printf("%d ",root->data);
}

int main(){
    struct Node *root=createNode(1);
    root->left=createNode(2);
    root->right=createNode(3);
    root->left->left=createNode(4);
    root->left->right=createNode(5);
    root->right->left=createNode(6);
    root->right->right=createNode(7);
    printf("Inorder Traversal: ");
    inorder(root);
    printf("\nPreorder Traversal: ");
    preorder(root);
    printf("\nPostorder Traversal: ");
    postorder(root);
    return 0;
}
```

OUTPUT:



Q12. Given a binary tree, the task is to determine the diameter of the tree. The diameter/width of a tree is defined as the number of edges on the longest path between any two nodes.

A12. CODE:

```
#include <stdio.h>
#include <stdlib.h>

struct Node{
   int data;
   struct Node *left,*right;
};

struct Node* createNode(int data){
   struct Node *newNode=(struct Node*)malloc(sizeof(struct Node));
   newNode->data=data;
   newNode->left=NULL;
   newNode->right=NULL;
   return newNode;
}

int max(int a,int b){{return a>b?a:b;}}
int diameter=0;
int height(struct Node *root){
   if(root==NULL){return 0;}
   int lh=height(root->left);
   int rh=height(root->right);
   if(lh+rh>diameter){diameter=lh+rh;}
   return max(lh,rh)+1;
}

int main(){
   struct Node *root=createNode(1);
   root->left=createNode(2);
   root->right=createNode(3);
   root->left->left=createNode(4);
   root->left->right=createNode(5);
   root->left->left->left=createNode(6);
   root->right->right=createNode(7);
   height(root);
   printf("Diameter of the tree: %d\n",diameter);
   return 0;
}
```

OUTPUT:

